```
« truncated »

Finished in 0.00762 seconds (files took 0.12669 seconds to load)
4 examples, 4 failures

Failed examples:

rspec ./spec/unit/app/api_spec.rb:31 # ExpenseTracker::API POST /expenses ↩
when the expense is successfully recorded responds with a 200 (OK)
rspec ./spec/unit/app/api_spec.rb:24 # ExpenseTracker::API POST /expenses ↩
when the expense is successfully recorded returns the expense id
rspec ./spec/unit/app/api_spec.rb:53 # ExpenseTracker::API POST /expenses ↩
when the expense fails validation responds with a 422 (Unprocessable entity)
rspec ./spec/unit/app/api_spec.rb:46 # ExpenseTracker::API POST /expenses ↩
when the expense fails validation returns an error message

Randomized with seed 8060
```

The error message has changed to `Wrong number of arguments. Expected 0, got 1.` RSpec sees the new `record` method, but points out that it doesn't take an argument like the test double does.

Make the method signatures match by adding a parameter:

05-unit-specs/10/expense_tracker/app/ledger.rb

```
def record(expense)
end
```

When you run the specs again, they should pass.

```
$ bundle exec rspec spec/unit/app/api_spec.rb
« truncated »

Finished in 0.0266 seconds (files took 0.13459 seconds to load)
4 examples, 0 failures

Randomized with seed 13686
```

A mock object guiding the interface for a real one—how's that for life imitating art? Now that you've got green specs, commit your work and take a well-deserved break. In the next chapter, you'll fill in the behavior behind the interface.

Your Turn

In this chapter, you've clarified exactly how your API is supposed to behave. You've spelled out what happens when storing an expense record succeeds or fails, using a test double to stand in for the unwritten persistence layer. You've refactored your specs so that they spell out exactly what behavior you're testing. Finally, you've used the design suggested by your tests to guide the interface of a real object.